ASSSIGNMENT NO: 8

NAME : PAYAL BHAGVAN CHARVANDE
PRN NO: 125B2F002
SUBJECT : DATABASE MANAGEMENT SYSTEM
DIV: IT A

Aim:

NoSQL: Aggregation and Indexing – Design and Develop Queries using Aggregation and Indexing with suitable example using MongoDB

Theory

1. Aggregation in MongoDB

Aggregation is used to process multiple documents and return computed results.
It works like SQL `GROUP BY`, `SUM`, `AVG`, `COUNT`, etc.

Common Aggregation Stages:

- `$match` → Filters documents
- `$group` → Groups data
- `$sum` → Calculates total
- `$avg` → Calculates average
- `$sort` → Sorts results
- `$project` → Displays selected fields

Advantages:

- Fast data analysis
- Performs calculations on grouped records
- Useful for reports and summaries

2. Indexing in MongoDB

Indexing improves query performance by reducing search time.

Without indexing:
→ MongoDB scans entire collection

With indexing:
→ MongoDB directly locates required documents

Types of Indexes:

1. Single Field Index
2. Compound Index
3. Unique Index
4. Text Index

Advantages:

- Faster searching
- Better sorting performance
- Optimized query execution

MongoDB uses `createIndex()` to create indexes.

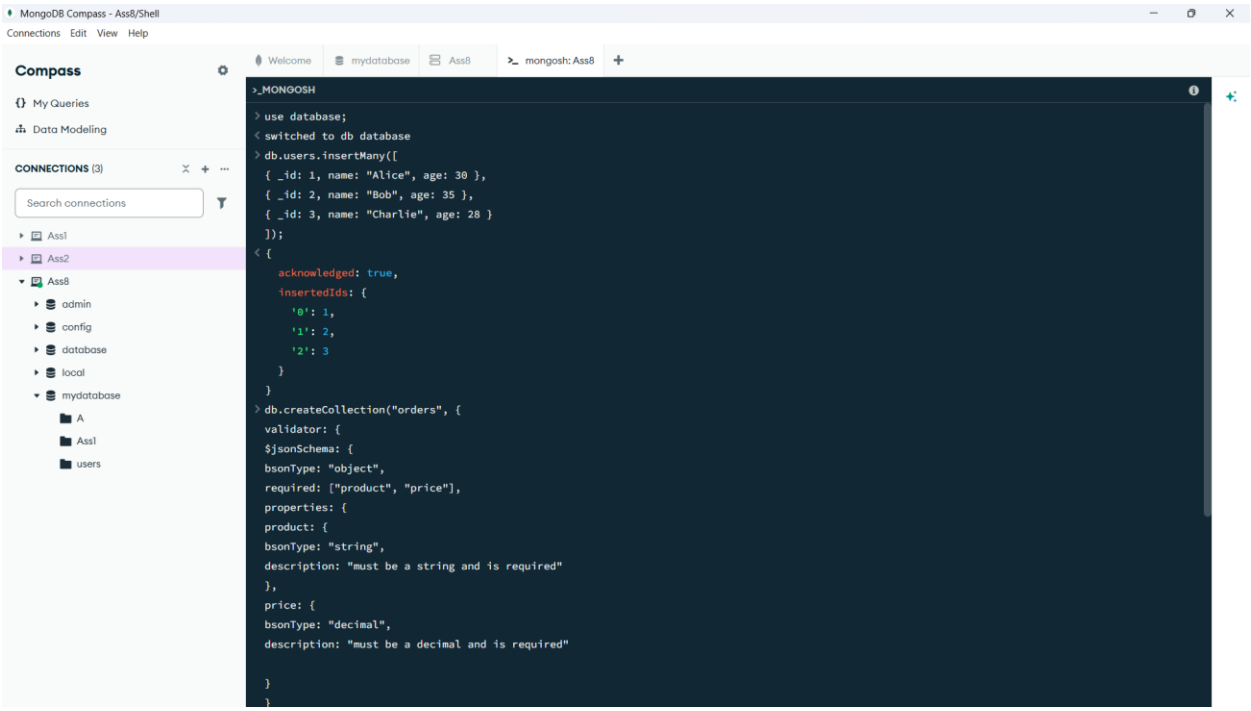
MONGODB

Step1: install using below link--->https://www.mongodb.com/try/download/community

Step2: add environment variable upto bin folder

Step3: Create New Connection bydefault give Name

Step4: execute below commands



use mydatabase;

switched to db mydatabase

```
db.users.insertMany([
{ _id: 1, name: "Alice", age: 30 },
{ _id: 2, name: "Bob", age: 35 },
{ _id: 3, name: "Charlie", age: 28 }
]);
{
acknowledged: true,
insertedIds: {
'0': 1,
'1': 2,
'2': 3
}
}
db.createCollection("orders", {
validator: {
$jsonSchema: {
bsonType: "object",
required: ["product", "price"],
properties: {
product: {
bsonType: "string",
description: "must be a string and is required"
},
price: {
bsonType: "decimal",
description: "must be a decimal and is required"

}
}
}
```

```
}
});
{ ok: 1 }
db.orders.insertOne({ price: 50.0 });
MongoServerError: Document failed validation
db.users.remove({ name: "Charlie"});
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete,
or bulkWrite.
{
acknowledged: true,
deletedCount: 1
}
db.users.remove({});
{
acknowledged: true,
deletedCount: 2
}
db.users.update(
{ _id: 1 },
{ $set: { name: &quot;Alicia&quot;, age: 31 } }
);
SyntaxError: Unexpected token (3:16)

1 | db.users.update(
2 | { _id: 1 },
> 3 | { $set: { name: &quot;Alicia&quot;, age: 31 } }
| ^
4 | );
db.users.update(
{ _id: 1 },
```

```
{ $set: { name: "Alicia", age: 31 } }
);

DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
acknowledged: true,
insertedId: null,
matchedCount: 0,
modifiedCount: 0,
upsertedCount: 0
}
db.users.update({ _id: 2 }, { $inc: { age: 1 } });
{
acknowledged: true,
insertedId: null,
matchedCount: 0,
modifiedCount: 0,
upsertedCount: 0
}
db.users.update(
{ _id: 4 },
{ $set: { name: "Eva", age: 29 } },
{ upsert: true }
);
{
acknowledged: true,
insertedId: 4,
matchedCount: 0,
modifiedCount: 0,
upsertedCount: 1
```

```
}
db.users.update({ age: { $gt: 30 } }, { $set: { status: "Senior" } }, { multi: true });
{
acknowledged: true,
insertedId: null,
matchedCount: 0,
modifiedCount: 0,
upsertedCount: 0


}
db.users.find({});
{
_id: 4,
age: 29,
name: 'Eva'
}
db.users.insertMany([
{ _id: 1, name: "Nikita", age: 30 },
{ _id: 2, name: "Vedant", age: 35 },
{ _id: 3, name: "Rushi", age: 28 }
]);
{
acknowledged: true,
insertedIds: {
'0': 1,
'1': 2,
'2': 3
}
}
mydatabase
```

Selection deleted

Conclusion:

This assignment demonstrates MongoDB Aggregation and Indexing successfully.
Aggregation helps analyze grouped data efficiently, while indexing improves query speed and performance. Both are essential for handling large datasets in NoSQL databases.